

Ex No: 3a

IMPLEMENTATION OF BLOCKS WORLD PROGRAM

Aim:

To implement binary search tree.

Procedure:

Initialize the BlocksWorld class to place each block into its own stack on the table.

Define display_state to print the current stack arrangement and find_block to locate which stack contains a specific block.

— —
Implement move to locate source and destination stacks, remove the block from its origin, append it to the target stack, and display the updated configuration.

Define goal_state to reassign the internal state to match a target stack configuration and display it.

Construct a 3-block world in main, set the goal state, and execute block relocation commands using move.

Program:

Class representing the state of the blocks world

class BlocksWorld:

def __init__(self, num_blocks):

Initialize blocks world with num_blocks on the table

self.state = [[block] for block in range(num_blocks)] # Each block starts on the table

self.num_blocks = num_blocks

def display_state(self):

Display the current state of the blocks world

for stack in self.state:

print(f'Block(s) on stack: {stack}')

def move(self, block, destination):

Move block onto another block or the table

Find the block in the current state and move it

source_stack = self.find_block(block)

```

destination_stack = self.find_block(destination)

if source_stack is None or destination_stack is None:
    print(f"Invalid block {block} or destination {destination}.")
    return

# Remove the block from its current position
source_stack.remove(block)

# Put the block on the destination stack
destination_stack.append(block)

# Display the state after the move
self.display_state()

def find_block(self, block):
    # Find the stack that contains the given block
    for stack in self.state:
        if block in stack:
            return stack
    return None

def goal_state(self, goal):
    # Goal state is a list of stacks that represents the desired configuration
    self.state = goal
    print("Goal state set.")
    self.display_state()

# Main program to solve Blocks World problem
def main():
    # Initialize the Blocks World with 3 blocks (0, 1, 2)

```

```
blocks_world = BlocksWorld(3)

print("Initial state:")
blocks_world.display_state()

# Define the goal state (Block 0 is on top of Block 1, and Block 2 is on the table)
goal = [[0, 1], [2]] # Goal configuration
blocks_world.goal_state(goal)

print("\nPerforming  Moves:")
# Move Block 0 onto Block 2
blocks_world.move(0, 2)

# Move Block 1 onto Block 2
blocks_world.move(1, 2)

# Move Block 2 onto the table (empty stack)
blocks_world.move(2, 0)

# Run the main program
if __name__ == "__main__":
    main()
```

Output:

Initial state:

Block(s) on stack: [0]

Block(s) on stack: [1]

Block(s) on stack: [2]

Goal state set.

Block(s) on stack: [0, 1]

Block(s) on stack: [2]

Performing Moves:

Block(s) on stack: [1]

Block(s) on stack: [2, 0]

Block(s) on stack: []

Block(s) on stack: [1]

Block(s) on stack: [2, 0]

Block(s) on stack: []

Block(s) on stack: [1, 2]

Block(s) on stack: [0]

Block(s) on stack: []

```
IDLE Shell 3.11.4
File Edit Shell Debug Options Window Help
Python 3.11.4 (tags/v3.11.4:8e8a0a4, Jun 6 2023, 13:03:39) [MSC v.1934 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>> ===== RESTART: C:/Users/Param/Desktop/blocks_world.py =====
Initial state:
Block(s) on stack: [0]
Block(s) on stack: [1]
Block(s) on stack: [2]
Goal state: [[0, 1, 2]]

Performing Moves:
Block(s) on stack: [1]
Block(s) on stack: [2, 0]
Block(s) on stack: [2, 0, 1]

>>> |
```